Image Processing

Rakshya Lama Moktan • 8 Jul (Edited 8 Jul)

Assignments • 100 points | Due 16 Jul, 20:45

Context

You are a computer vision engineer at FreshTrack, a produce supply chain company. Warehouses receive mixed crates of fruit and need to automatically sort them by type, flag color defects (underripe or overripe), and count individual pieces before packing. Manual inspection is slow and inconsistent under variable warehouse lighting. Your job is to build a CV pipeline using OpenCV that can: (1) identify fruit types by color, (2) clean up noisy camera images, (3) detect fruit shape features, and (4) count individual round fruits in a batch image.

Images to Use

- red_apple.jpg, green_apple.jpg, banana.jpg, strawberry.jpg, orange.jpg, lime.jpg — individual fruit images (Fruits-360 dataset subset)
- mixed_bowl.jpg — a single image containing multiple fruits; used for the full pipeline and counting tasks
- morphology.png — binary morphology demo image
- chessboard.png — high-contrast grid for Harris corner calibration
- All images provided below

Part A — Color Space & Fruit Segmentation

1. Load red_apple.jpg in BGR using cv2.imread(). Display it correctly in Matplotlib (convert to RGB first). Then display the raw BGR version. What do you notice about the colors?
2. Convert red_apple.jpg to HSV. Visualize the H, S, and V

channels side by side as grayscale images. Which channel best separates the fruit from its white background?

3. Masking by color: use cv2.inRange() in HSV to isolate the red apple. Red wraps around the Hue axis (0–10 and 170–180). Create two masks and combine with cv2.bitwise_or(). Display: original | mask | masked result.

4. Color table task: repeat the HSV masking for green_apple.jpg, banana.jpg, and lime.jpg. Record each fruit's working HSV range as a markdown table: Fruit | H_min | H_max | S_min | S_max | V_min | V_max.

5. Apply manual brightness/contrast to a dark banana image (alpha=1.3, beta=20) using NumPy. Clip to [0,255] and convert to uint8. Compare to cv2.convertScaleAbs(). Does it make the HSV mask more reliable?

6. Reflection: why does HSV work better than RGB for fruit color segmentation? When would it fail? (Hint: try a very dark purple grape or blueberry.)

Part B — Histograms & Filtering

1. Load strawberry.jpg as grayscale. Plot its histogram. Is the image well-exposed? What does the histogram shape tell you about contrast?
2. Apply global histogram equalization (cv2.equalizeHist()). Then apply CLAHE with clipLimit=2.0 and clipLimit=8.0. Show all four versions side by side (original, equalized, CLAHE-2, CLAHE-8). Which helps the HSV segmentation mask most?
3. Noise experiment: add Gaussian noise (mean=0, std=25) to orange.jpg using np.random.normal(). Apply Gaussian blur, median filter, and bilateral filter. Which removes the noise best while keeping the fruit's edge sharp? Plot all four for comparison.
4. Extended — manual convolution: implement conv2d() from scratch. Apply a Sobel X kernel to the grayscale banana image. Compare to cv2.filter2D(). What is the mean absolute difference? Explain any discrepancy.

Part C — Morphology & Mask Cleanup

A raw HSV color mask is rarely
clean — it has small noise blobs from reflections and gaps from shadows. Morphological operations fix this before any shape analysis.
5. Take your banana HSV mask from Part A. Apply erosion
with a 5×5 kernel. What happens to the mask?
6. Apply opening (erosion then dilation) to remove noise.
Apply closing (dilation then erosion) to fill holes. Show all four: raw mask | after erosion | after opening | after closing.
7. Apply the morphological gradient (dilation – erosion)
to extract just the outline of the banana. Overlay it on the original image in green.
8. Reflection: what would happen if you applied
morphological operations directly to the BGR fruit image instead of the binary mask? Try it and describe what you see.
9. Demo with j.png: apply erosion, dilation, opening,
closing, Top Hat, and Black Hat. Show all six results. Which operation isolates the thin strokes?

Part D — Shape & Feature Detection

10. Canny
edge detection: run cv2.Canny() on grayscale green_apple.jpg with threshold pairs [30, 100] and [100, 200]. Show both results. Which captures the fruit outline cleanly without noise inside? Explain what happens at each stage of the Canny pipeline.
11. Extended
— Canny from scratch: use the provided CannyEdgeDetector class to run the full pipeline step by step on the same image (Gaussian blur → gradient

magnitude/direction → NMS → double threshold → hysteresis). Compare to cv2.Canny().

12. Harris

corners on chessboard.jpg: run cv2.cornerHarris() and dilate the response map. Threshold and mark corners as red dots. Adjust the response threshold from 0.01 to 0.1 — what changes?

13. Hough

Circle Transform on mixed_bowl.jpg: run cv2.HoughCircles() with minRadius=30, maxRadius=100. Tune param2 (accumulator threshold) until you detect the round fruits cleanly. Draw circles and center points. How many round fruits does your detector count? Does it match the actual number?

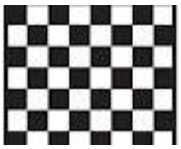
14. Full

pipeline: for one fruit of your choice from mixed_bowl.jpg, run the complete pipeline: load → BGR-to-RGB display → HSV mask → morphological cleanup → Canny edges → draw bounding box → save with safe_imwrite(). This is your final deliverable for Part D.

Submit --> notebook file (.ipynb or .pdf) with all outputs visible

mixed_fruit_bowl.jpeg
Image

chessboard.png
Image



Fruits-360 dataset
https://www.kaggle.com/datasets/n

morphology.png
Image

Your work

Assigne

+ Add or create

Mark as Done

Private comments

Add comment to Rakshya Lam...

Class comments

Add comment